Welcome

Digital and Computer Vision

Lesson 3: Session 1

Image segmentation and object extraction

Summary of the **previous** session: Thresholding , Edge Detection & Contours.

In the previous session we have covered :

Detecting edges using Sobel, Laplacian, and Canny edge detection.
Finding and drawing contours in images.
Tresholding, convex hulls and shape approximations.
Hough Transform

Summary of the current session: Image segmentation and object extraction

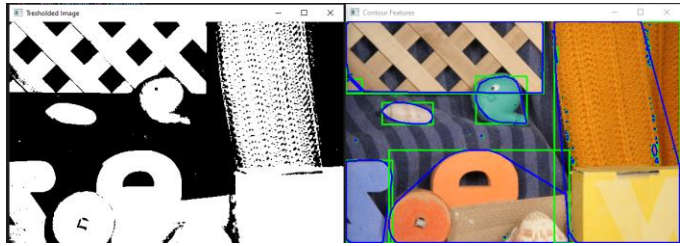
In this session we will covered the following:

Watershed algorithm for image segmentation.
K-means clustering for color segmentation.
GrabCut for object extraction

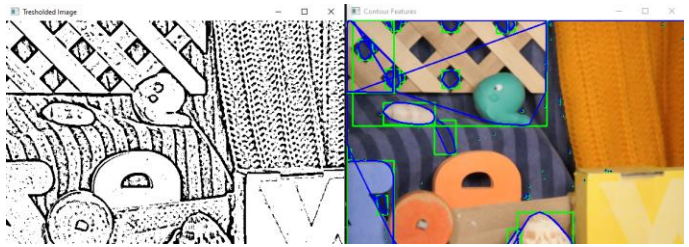
Exercise : from previous session

Test contour detection and drawing features on images rubberwhale1.png and stuff.jpg. Use different thresholding methods and edge detection for binarization of the images. Discuss the obtained results.

```
cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)
```

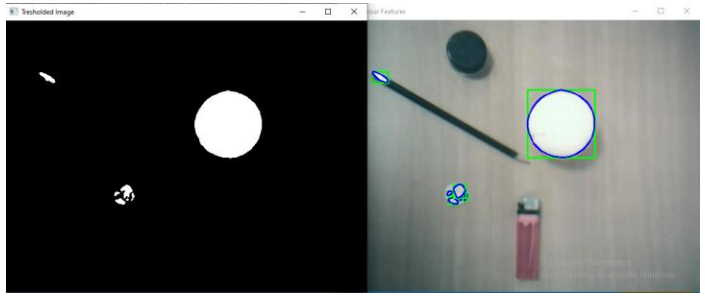


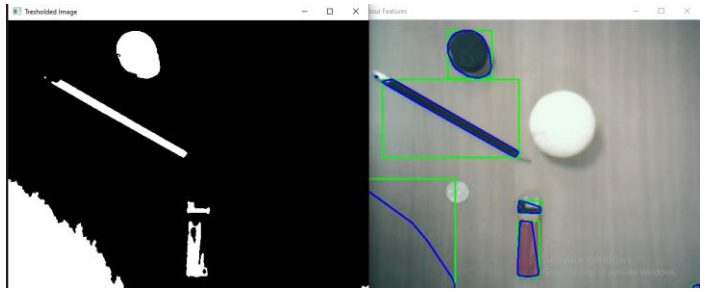

```
thresh = cv2.adaptiveThreshold(gray,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,\
        cv2.THRESH_BINARY,11,2)
```



Exercise : from previous session code example

_, thresh = cv2.threshold(gray, 190, 255, cv2.THRESH_BINARY) and cv2.THRESH_BINARY_INV





thresh = cv2.Canny(gray, 20, 200)

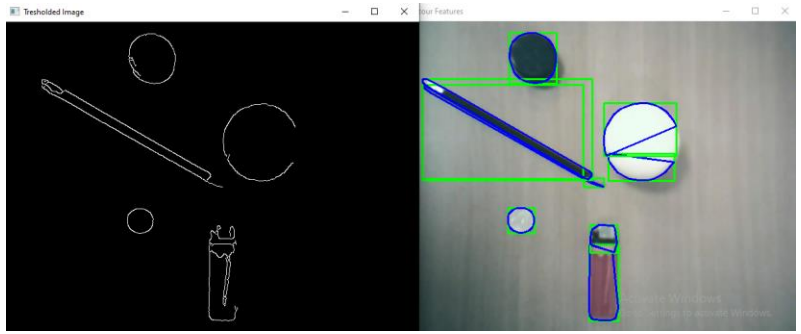
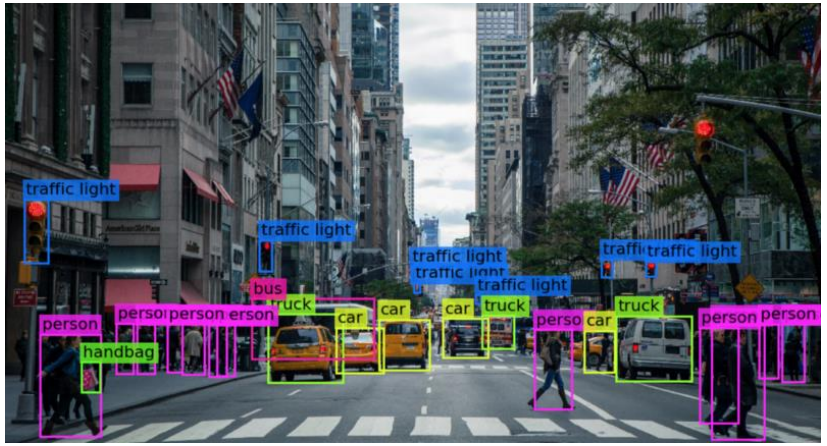


Image Segmentation

Image segmentation is a computer vision technique used to divide an image into multiple segments or regions that have strong correlations. The goal is to identify objects, boundaries, or specific areas of the real world within an image. It is widely used in fields like medical imaging, autonomous driving, facial recognition.

Complete segmentation : which results in a set of disjoint regions corresponding uniquely with objects on the image
Partial segmentation : in which regions do not corresponding directly to objects on the image



Types of algorithms for Image Segmentation

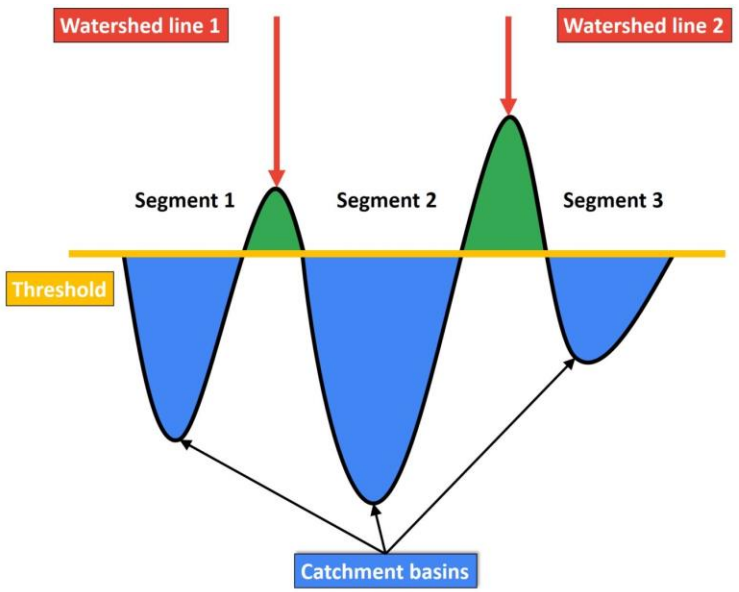
1. **Thresholding-Based Segmentation** : Uses intensity values to separate regions.
2. **Edge-Based Segmentation** : Detects object boundaries by identifying strong intensity changes.
3. **Region-Based Segmentation** : Groups pixels with similar properties (e.g., color, texture).
4. **Clustering-Based Segmentation** : Uses algorithms like K-means or Gaussian Mixture Models (GMM) to classify pixels into clusters.
5. **Deep Learning-Based Segmentation** : Uses convolutional neural networks (CNNs) for more accurate segmentation.

Watershed algorithm for image segmentation

The Watershed Algorithm is a region-based image segmentation technique used to separate overlapping or touching objects in an image. It is especially useful for medical imaging, object recognition, and image processing applications.

The algorithm is based on the topographic representation of an image, treating pixel intensity as a height map, where high-intensity pixels act like peaks and low-intensity pixels act like valleys.

Any grayscale image can be viewed as a topographic surface where high intensity denotes peaks and hills while low intensity denotes valleys. You start filling every isolated valleys (local minima) with different colored water (labels). As the water rises, depending on the peaks (gradients) nearby, water from different valleys, obviously with different colors will start to merge. To avoid that, you build barriers in the locations where water merges. The barriers you created gives you the segmentation result.



https://datahacker.rs/007-opencv-projects-image-segmentation-with-watershed-algorithm/

How Does the Watershed Algorithm Work?

If we flood this surface from its minima and, if we prevent the merging of the waters coming from different sources, we partition the image into two different sets: the catchment basins and the watershed lines. A major enhancement of the watershed transformation consists in flooding the topographic surface from a previously defined set of markers. Segmenting an image by the watershed transformation is therefore a two-step process: Finding the markers and performing a marker-controlled watershed.

1. Convert the image to grayscale (if it's not already).
2. Apply thresholding or edge detection to identify objects in the image.
3. Compute distance transform to identify object centers.
4. Identify markers (foreground & background).
5. Apply the Watershed algorithm.
6. Objects are separated based on marker labels.

Distance Transform

The distance transform helps find the center of objects by calculating the shortest distance of each pixel from the nearest zero pixel (background). It highlights object centers, which helps in detecting separate objects. It outputs a distance map or distance matrix where each cell matches a pixel of the input image and contains a value indicating the computed distance.

0	0	0	0	0	0	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	1	1	1	1	1	0
0	0	0	0	0	0	0

Binary Image

0	0	0	0	0	0	0
0	1	1	1	1	1	0
0	1	2	2	2	1	0
0	1	2	3	2	1	0
0	1	2	2	2	1	0
0	1	1	1	1	1	0
0	0	0	0	0	0	0

Distance transformation

https://en.wikipedia.org/wiki/Distance_transform

1. City Block Distance (Manhattan Distance): Uses only horizontal and vertical neighbors.
2. Chessboard Distance.
3. Euclidean Distance.

Define Markers (Foreground & Background)

In image processing, markers are special labels used to define regions of interest (ROIs) in an image. These markers help algorithms like the Watershed Algorithm and Connected Components Analysis distinguish between different objects and the background.

Markers are used to label foreground objects, background areas, and uncertain regions.

1. Foreground markers (sure objects)
2. Background markers (sure background)
3. Unknown region (where objects touch)

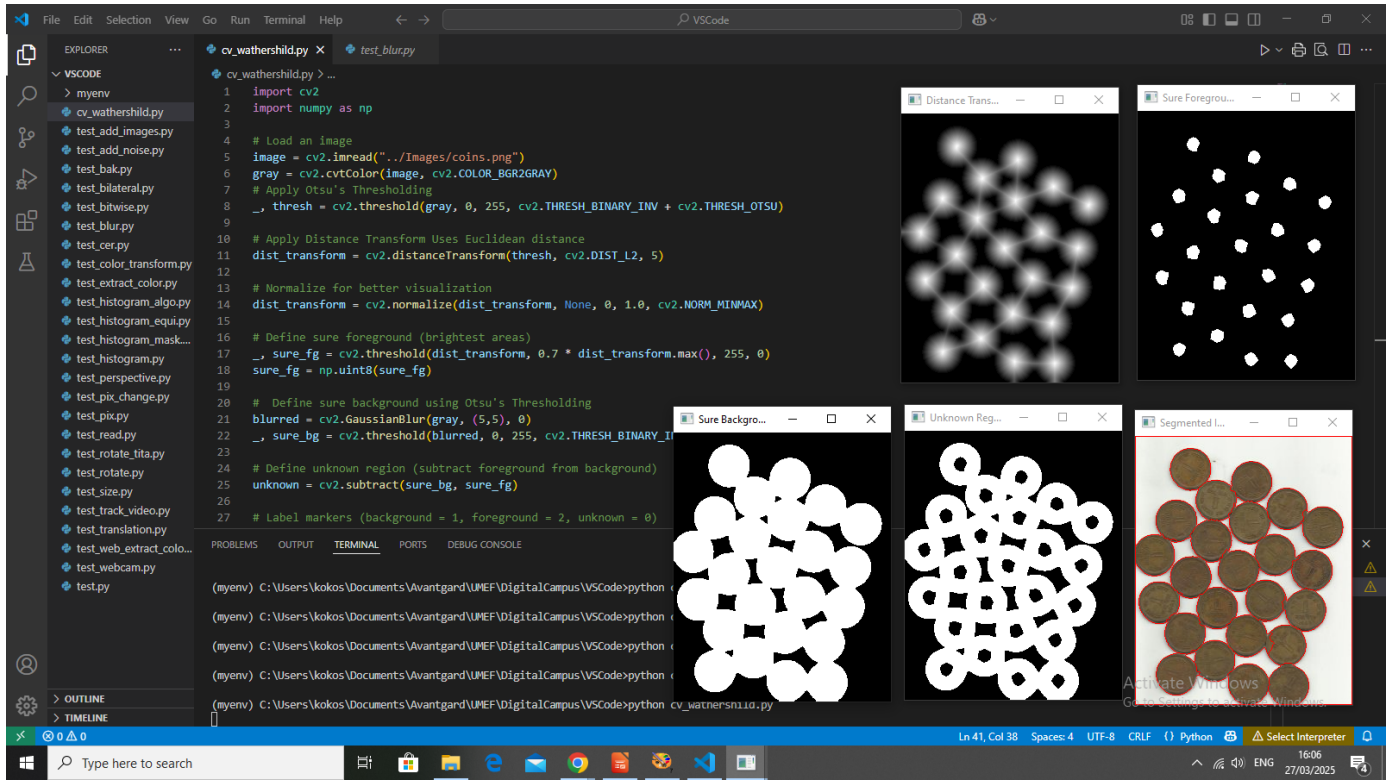
Watershed algorithm – code example 1/3

```
import cv2
import numpy as np

# Load an image
image = cv2.imread("../Images/coins.png")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply Otsu's Thresholding
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# Apply Distance Transform Uses Euclidean distance
dist_transform = cv2.distanceTransform(thresh, cv2.DIST_L2, 5)

# Normalize for better visualization
dist_transform = cv2.normalize(dist_transform, None, 0, 1.0, cv2.NORM_MINMAX)

# Define sure foreground (brightest areas)
_, sure_fg = cv2.threshold(dist_transform, 0.7 * dist_transform.max(), 255, 0)
sure_fg = np.uint8(sure_fg)
```

Watershed algorithm – code example 2/3

```
#  Define sure background using Otsu's Thresholding
blurred = cv2.GaussianBlur(gray, (5,5), 0)
_, sure_bg = cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# Define unknown region (subtract foreground from background)
unknown = cv2.subtract(sure_bg, sure_fg)

# Label markers (background = 1, foreground = 2, unknown = 0)
#  Detects connected regions and assigns unique labels.
#  It labels background of the image with 0, then other objects are labelled with integers starting from 1.
markers = cv2.connectedComponents(sure_fg)[1]
markers = markers + 1  # Ensure background is not 0
markers[unknown == 255] = 0  # Mark unknown region as 0

# Apply Watershed algorithm
cv2.watershed(image, markers)
```

Watershed algorithm – code example 3/3




```
# Mark detected object boundaries in RED
image[markers == -1] = [0, 0, 255]

cv2.imshow("Segmented Image", image)
cv2.imshow("Sure Foreground", sure_fg)
cv2.imshow("Sure Background", sure_bg)
cv2.imshow("Unknown Region", unknown)
cv2.imshow("Distance Transform matrix", dist_transform)
cv2.waitKey(0)
```

Connected Component Analysis

Connected Component Analysis (CCA) is a technique used in image processing to label and identify separate objects in a binary image. It groups together connected pixels that share the same properties (e.g., white pixels in a binary image).

Connected components can be identified based on 4-connectivity or 8-connectivity:
4-Connectivity → Pixels are considered connected if they share a side (left, right, up, or down).
8-Connectivity → Pixels are connected if they share a side or a corner (diagonal connectivity).

Why Use Connected Components?
1. **Object Counting** → Count the number of objects in an image.
2. **Object Segmentation** → Identify and separate objects from the background.
3. **Feature Extraction** → Measure the size, shape, and position of objects.
4. **Preprocessing for Watershed Algorithm** → Helps in defining foreground/background markers.

How Does Connected Component Algorithm Work ?

In short, once the first pixel of a connected component is found, all the connected pixels of that connected component are labelled before going onto the next pixel in the image.

Before we apply connected component analysis, the image must be thresholded to a binary form (black and white).

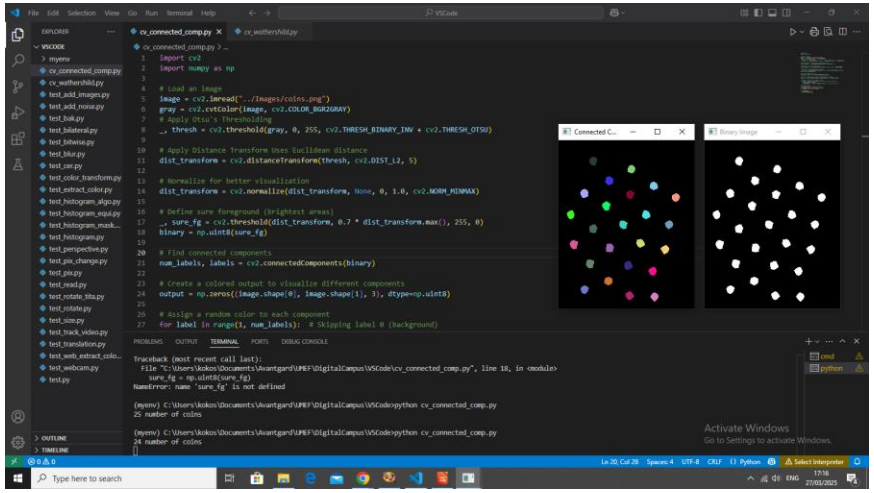
1. Start from the first pixel in the image. Set current label to 1. Go to (2).
2. If this pixel is a foreground pixel and it is not already labelled, give it the current label and add it as the first element in a queue, then go to (3). If it is a background pixel or it was already labelled, then repeat (2) for the next pixel in the image.
3. Pop out an element from the queue, and look at its neighbours (based on any type of connectivity). If a neighbour is a foreground pixel and is not already labelled, give it the current label and add it to the queue. Repeat (3) until there are no more elements in the queue.
4. Go to (2) for the next pixel in the image and increment current label by 1.

Connected Component code example 1/2

```
import cv2
import numpy as np

# Load an image
image = cv2.imread("../Images/coins.png")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply Otsu's Thresholding
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# Apply Distance Transform Uses Euclidean distance
dist_transform = cv2.distanceTransform(thresh, cv2.DIST_L2, 5)

# Normalize for better visualization
dist_transform = cv2.normalize(dist_transform, None, 0, 1.0, cv2.NORM_MINMAX)

# Define sure foreground (brightest areas)
_, sure_fg = cv2.threshold(dist_transform, 0.7 * dist_transform.max(), 255, 0)
binary = np.uint8(sure_fg)
```

Connected Component code example 2/2

```
# Find connected components
num_labels, labels = cv2.connectedComponents(binary)

# Create a colored output to visualize different components
output = np.zeros((image.shape[0], image.shape[1], 3), dtype=np.uint8)

# Assign a random color to each component
for label in range(1, num_labels):  # Skipping label 0 (background)
    mask = labels == label
    output[mask] = np.random.randint(0, 255, 3)  # Assign random color

# Show the result
print(num_labels-1, "number of coins")
cv2.imshow("Connected Components", output)
cv2.imshow("Binary Image", binary)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



K-Means Clustering

K-Means is an unsupervised Machine Learning (ML) algorithm used for clustering similar data points together. In image processing, K-Means is widely used for color quantization and segmentation, where pixels with similar colors are grouped into clusters.

Supervised ML – use labeled data
Unsupervised ML – use non labeled data

Applications of K-Means in Image Processing :

1. Color-Based Segmentation – Extract objects of similar colors
2. Color Quantization – Reduce the number of colors in an image
3. Image Compression – Store images with fewer colors
4. Background Removal – Isolate foreground objects

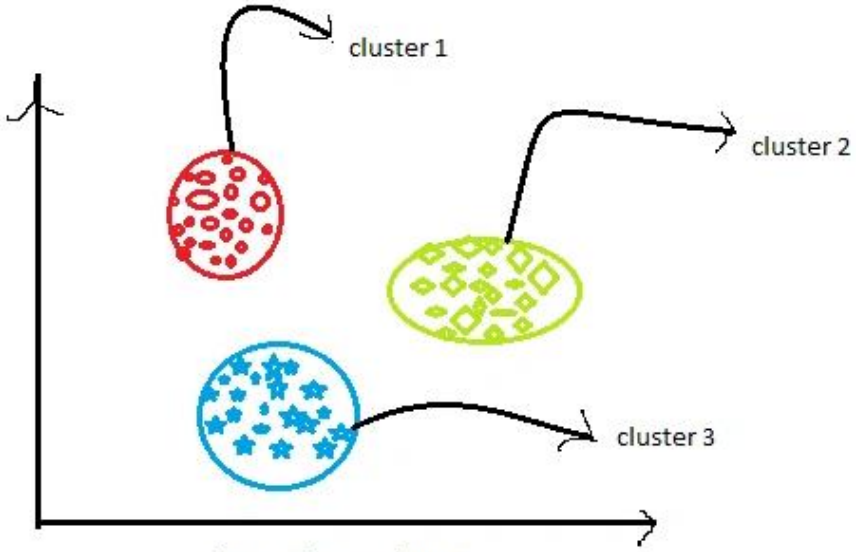


fig 2: After applying K-means clustering

https://www.analyticsvidhya.com/blog/2020/10/a-simple-explanation-of-k-means-clustering/

How Does K-Means Work?

The K-Means algorithm groups data into K clusters by minimizing the variance within each cluster.

1. Choose the Number of Clusters (K)
2. Initialize Cluster Centers (Centroids) : K initial cluster centers (centroids) are randomly chosen.
3. Assign Data Points to the Nearest Cluster : Each point is assigned to the nearest cluster centroid based on Euclidean distance:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

4. Update Centroids : Compute the new cluster centroids by averaging all points in each cluster.

$$C_j = \frac{1}{N_j} \sum_{i=1}^{N_j} X_i$$

where:

- C_j = new centroid of cluster j
- N_j = number of points in cluster j
- X_i = pixel values in cluster j

5. Repeat Until Convergence :

Centroids don't change.

The difference between iterations is below a threshold.

K-Means Clustering - Illustrative example

Consider, the t-shirt problem where you use height and weights of people to decide the size of t-shirt. In this case, we set a test data of size 50x2, which are heights and weights of 50 people. First column corresponds to height and second column corresponds to their weights. In ML we call these height and weight **features**. **Features** are measurable properties or characteristics of the data used for classification or prediction.

Cluster the data in 2 different sizes.

Features

	Height	Weight
Person 1	H1	W1
Person 2	H2	W2
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
Person 50	H50	W50

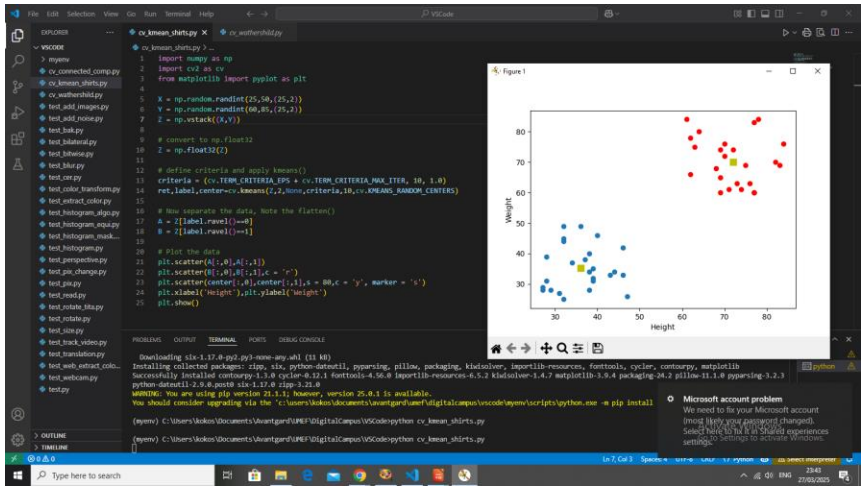
https://docs.opencv.org/4.x/d1/d5c/tutorial_py_kmeans_opencv.html

K-Means Clustering - Illustrative example code 1/2

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

X = np.random.randint(25,50,(25,2))
Y = np.random.randint(60,85,(25,2))
Z = np.vstack((X,Y))

# convert to np.float32
Z = np.float32(Z)

# define criteria and apply kmeans()
criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 10, 1.0)
ret,label,center=cv.kmeans(Z,2,None,criteria,10,cv.KMEANS_RANDOM_CENTERS)
```

K-Means Clustering - Illustrative example code 2/2

```
# Now separate the data, Note the flatten()
A = Z[label.ravel()==0]
B = Z[label.ravel()==1]

# Plot the data
plt.scatter(A[:,0],A[:,1])
plt.scatter(B[:,0],B[:,1],c = 'r')
plt.scatter(center[:,0],center[:,1],s = 80,c = 'y', marker = 's')
plt.xlabel('Height'),plt.ylabel('Weight')
plt.show()
```



K-Means Clustering for Color Segmentation

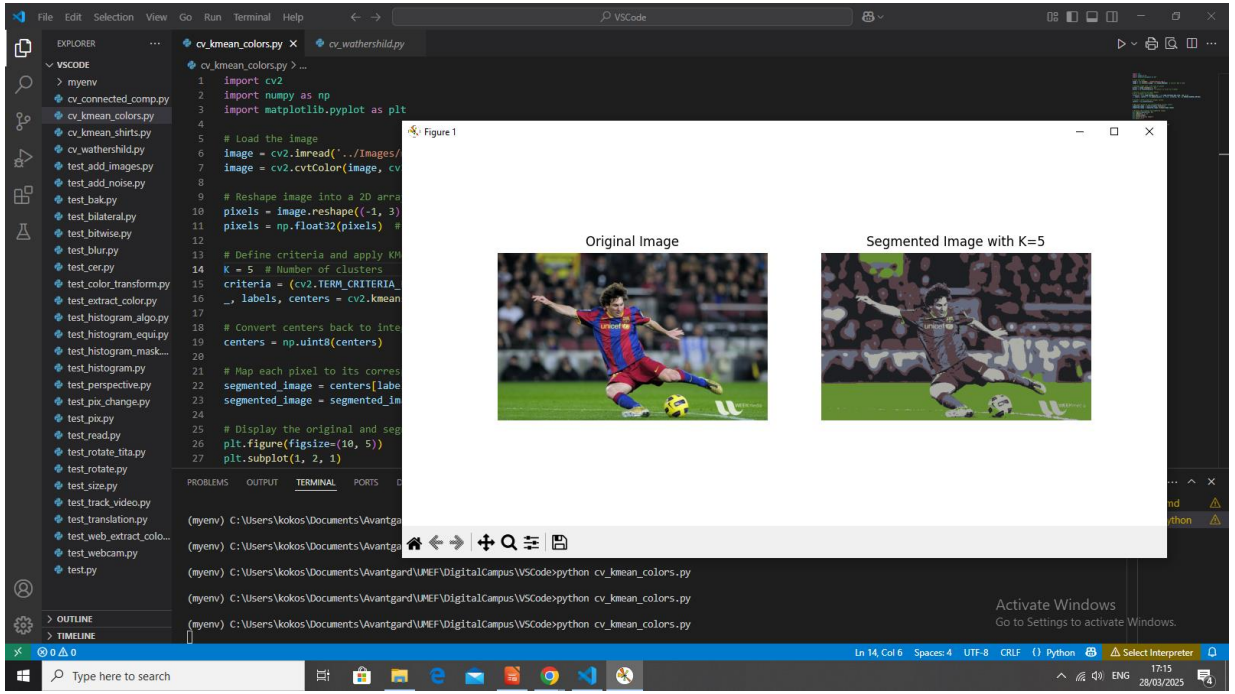
K-Means is widely used for color quantization and segmentation, where pixels with similar colors are grouped into clusters. An image consists of pixels, and each pixel has RGB (or BGR in OpenCV) values. Instead of treating the image as a grid, we reshape it into a list of pixels where each pixel is represented as a 3D vector: (R, G, B). Therefore we will have not 2 as the previous example but 3 features.

K-Means Clustering for Color Segmentation - example code 1/2

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image
image = cv2.imread('../Images/messi5.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)  # Convert BGR to RGB

# Reshape image into a 2D array of pixels
pixels = image.reshape((-1, 3))
pixels = np.float32(pixels)  # Convert to float for k-means

# Define criteria and apply KMeans
K = 5  # Number of clusters
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100,
0.2)
_, labels, centers = cv2.kmeans(pixels, K, None, criteria, 10,
cv2.KMEANS_RANDOM_CENTERS)
```

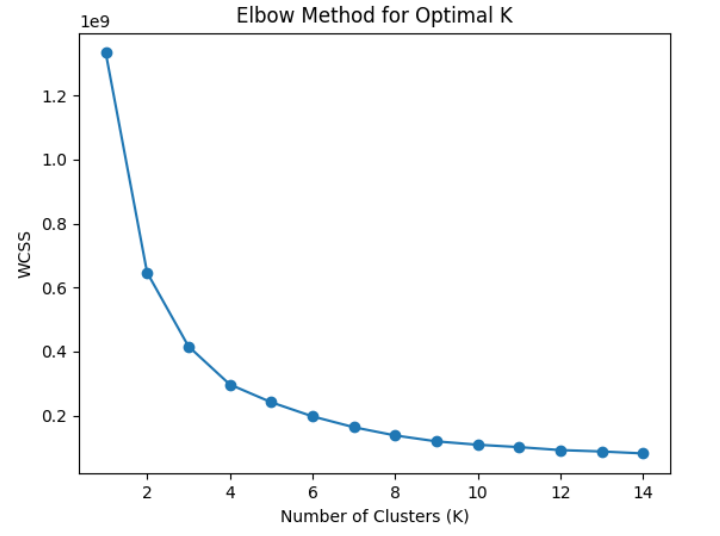
K-Means Clustering for Color Segmentation - example code 2/2

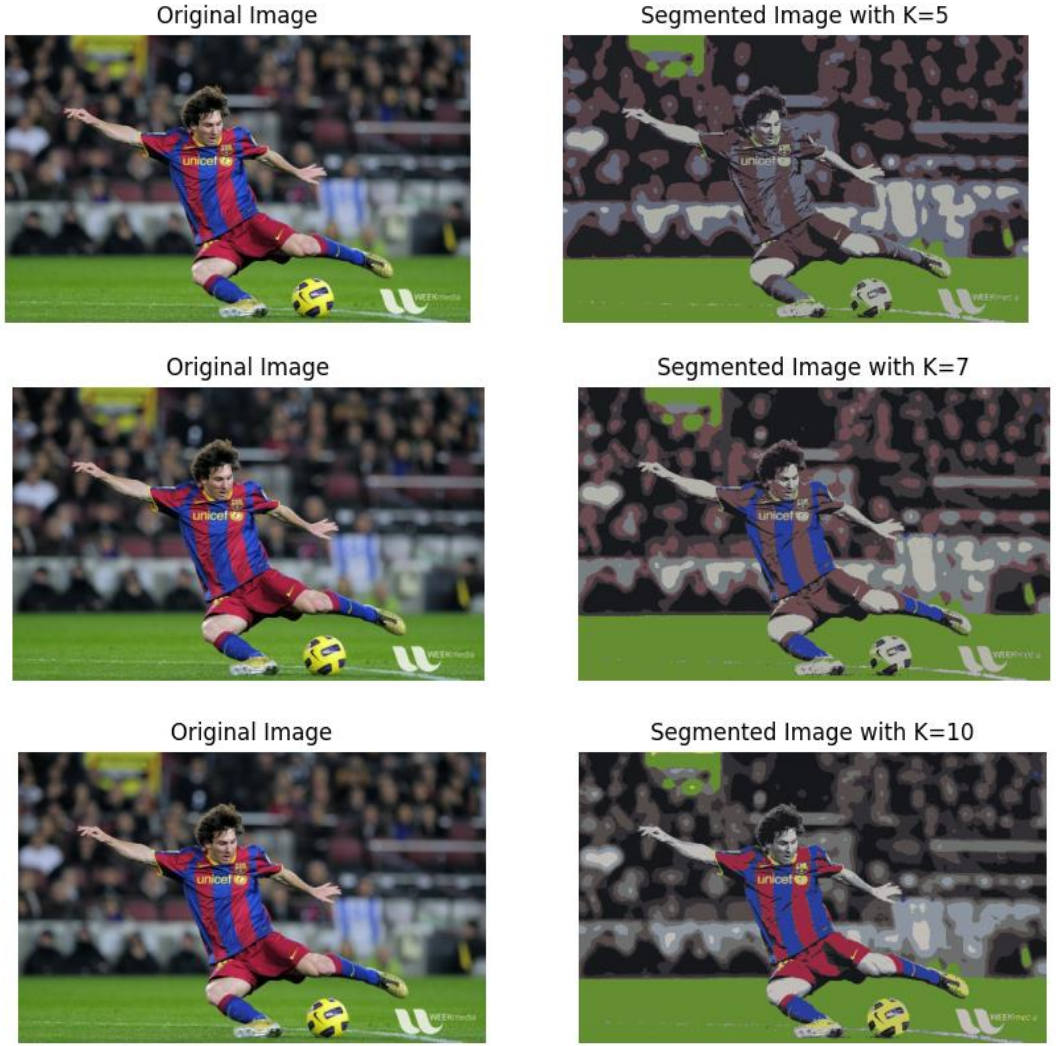
```
# Convert centers back to integer values
centers = np.uint8(centers)

# Map each pixel to its corresponding cluster center
segmented_image = centers[labels.flatten()]
segmented_image = segmented_image.reshape(image.shape)

# Display the original and segmented images
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(segmented_image)
plt.title("Segmented Image with K={}".format(K))
plt.axis("off")
```



K-Means Clustering – how to choose the number of clusters ?

One possibility is to simply check different numbers for best results. You can also use the within-cluster sum of squares (WCSS) and coose the K where the WCSS curve stop to change too much indicating convergence - the difference between iterations is low.





GrabCut for object extraction

GrabCut uses Graph Cut optimization and an iterative energy minimization approach to segment the image into:

1. Foreground (object)
2. Background
3. Probable foreground/background regions

It requires an initial user input (a rectangle or mask) to guide the segmentation.

Applications of GrabCut :

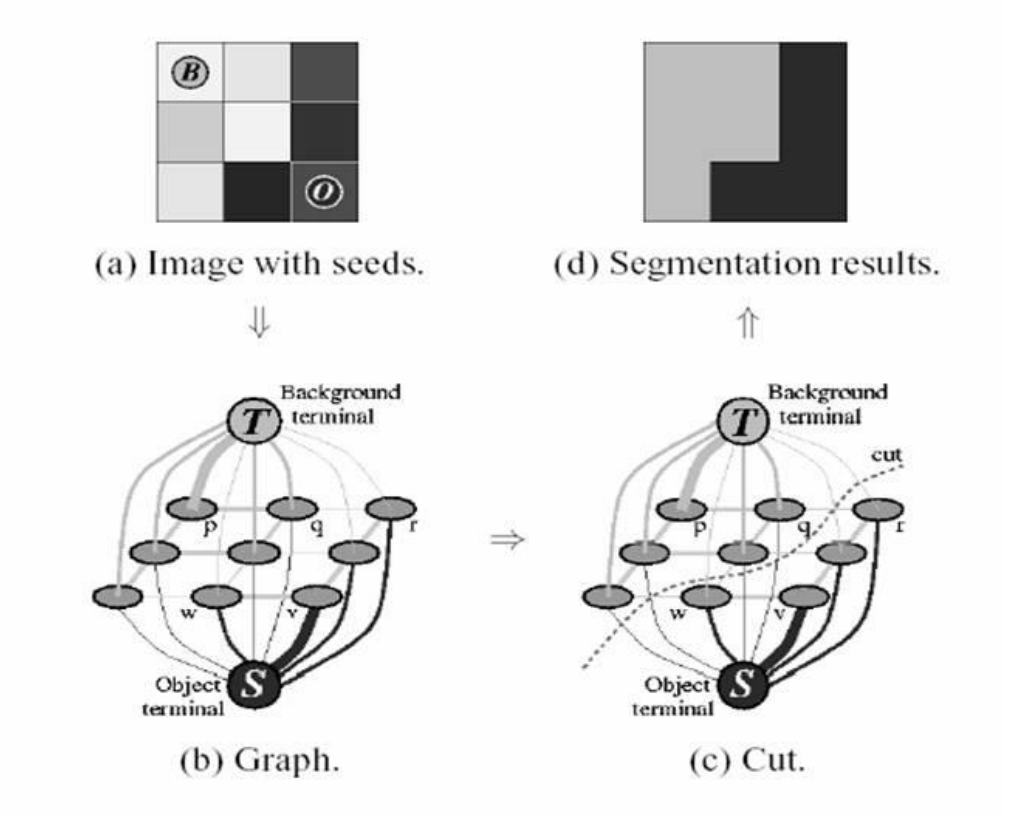
1. Object extraction from images.
2. Background removal.
3. Portrait cutouts.
4. Image compositing (e.g., adding a new background).

GrabCut Algorithm Steps

1. **User Input (ROI)** : The user marks a rectangle around the object in the image. This rectangle acts as a hint:
Everything outside the rectangle is background.
Everything inside is unknown (could be foreground or background).
2. **Initialization with GMM (Gaussian Mixture Model)** : Initializes two GMMs, for the foreground and the background. These GMMs help in modeling the color distributions of foreground and background pixels.
3. **Graph Construction (Min-Cut Algorithm)** : The image is converted into a graph, where:
Each pixel is a node.
Edges connect neighboring pixels.
Edge weights depend on color similarity and position.
4. **Energy Minimization** : The Graph Cut algorithm is used to classify pixels into foreground and background. It minimizes energy by finding the best cut between foreground and background. The classification is iteratively refined.
5. **User Refinement (Optional)** : Users can refine the mask to mark certain foreground or/and background pixels manually. Then the algorithm re-runs to improve segmentation.

GrabCut Algorithm

The figure below taken from https://www.cs.ru.ac.za/research/g02m1682/ shows the process of labelling an image, creating the graph, and then segmenting the graph to produce a segmentation of the image.



GrabCut in OpenCV

OpenCV has the function, cv.grabCut(). The arguments are:

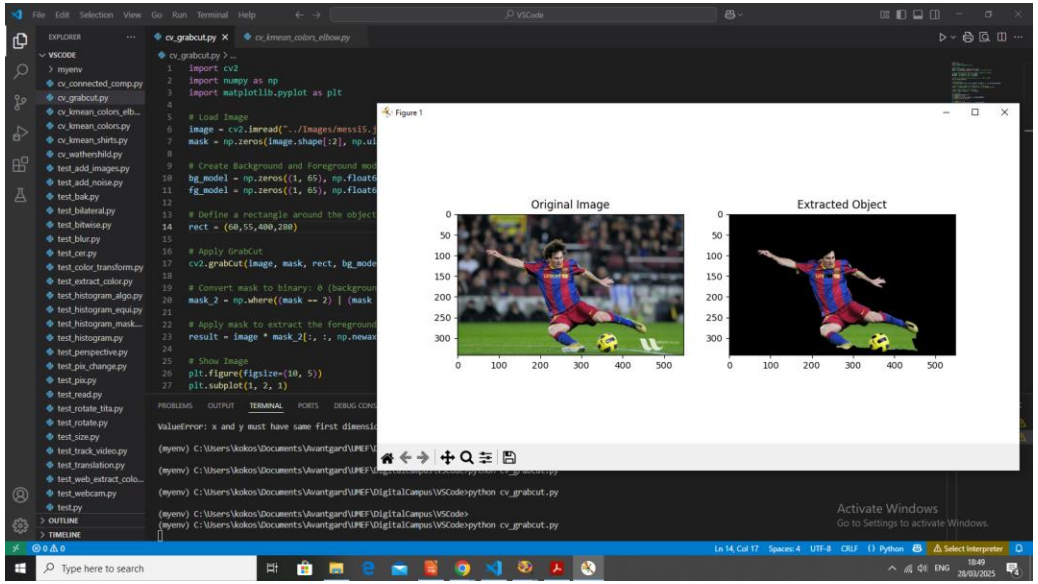
img - Input image
mask - It is a mask image where we specify which areas are background, foreground or probable background/foreground etc. It is done by the following flags, cv.GC_BGD, cv.GC_FGD, cv.GC_PR_BGD, cv.GC_PR_FGD, or simply pass 0,1,2,3 to image.
rect - It is the coordinates of a rectangle which includes the foreground object in the format (x,y,w,h)
bdgModel, fgdModel - These are arrays used by the algorithm internally. You just create two np.float64 type zero arrays of size (1,65).
iterCount - Number of iterations the algorithm should run.
mode - It should be cv.GC_INIT_WITH_RECT or cv.GC_INIT_WITH_MASK or combined which decides whether we are drawing rectangle or final touchup strokes.

GrabCut Algorithm code example 1/2

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load Image
image = cv2.imread("../Images/messi5.jpg")
mask = np.zeros(image.shape[:2], np.uint8)  # Initialize mask

# Create Background and Foreground models (needed for GrabCut)
bg_model = np.zeros((1, 65), np.float64)
fg_model = np.zeros((1, 65), np.float64)

# Define a rectangle around the object (x, y, width, height)
rect = (60,55,400,280)
```

GrabCut Algorithm code example 2/2

```
# Apply GrabCut
cv2.grabCut(image, mask, rect, bg_model, fg_model, 5, cv2.GC_INIT_WITH_RECT)

# Convert mask to binary: 0 (background), 1 (foreground)
mask_2 = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')

# Apply mask to extract the foreground
result = image * mask_2[:, :, np.newaxis]

# Show Image
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Extracted Object")
plt.show()
```



Exercises:

1. Modify the code of K-Means Clustering for Color Segmentation to compute the within-cluster sum of squares (WCSS) for K 1 to 15 and plot the results in a matplotlib.pyplot plot.

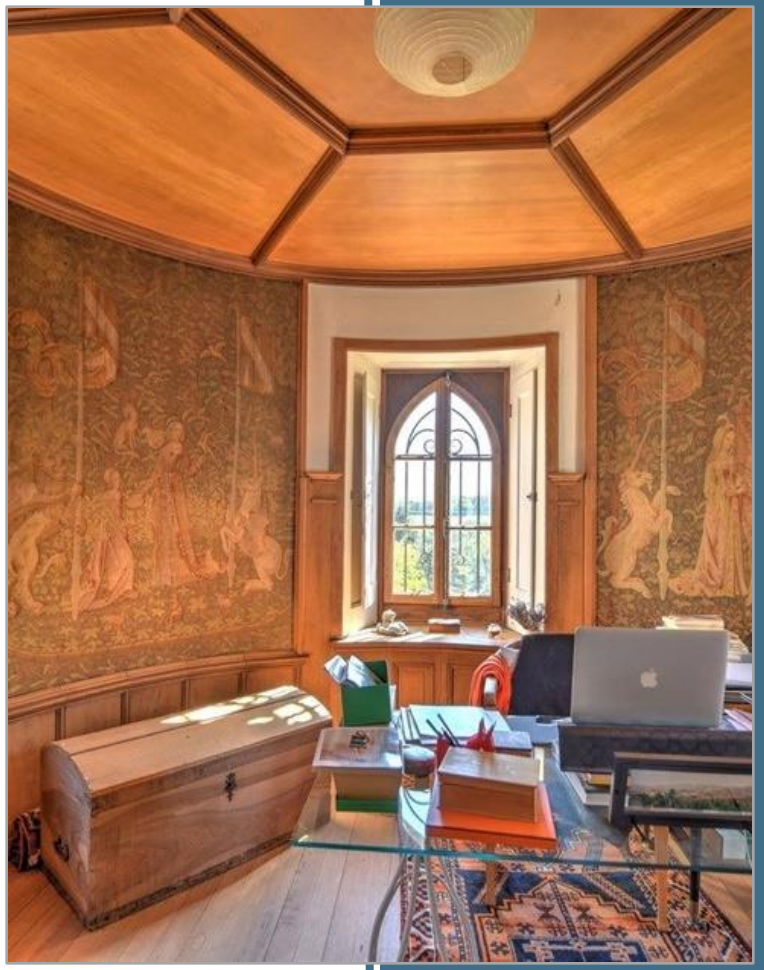
2. Get the interactive tool using grabcut, grabcut.py, from github. You can refine the grabCut result (step 5 of the algorithm) by use of this interactive tool.

https://github.com/opencv/opencv/blob/4.x/samples/python/grabcut.py

In this session we have covered:

Image segmentation and object extraction
Watershed algorithm for image segmentation.
K-means clustering for color segmentation.
GrabCut for object extraction

In the next session we will covered:

Feature Detection with OpenCV
What are features on images?
Detecting key points using Harris Corner Detection.
Feature matching (ORB, SIFT, and SURF).

THANKS !!!